Advanced Scheduling

2026 Semester 1 SOFTENG 370: Operating Systems

Lecture 9

Talia Xu

2.0.0

Based on original slides by Professor Jon Eyolfson.

How to schedule without perfect knowledge?

How can we design a scheduler system that both

- minimizes response time for interactive jobs
- minimizes turnaround time without a priori knowledge of job length

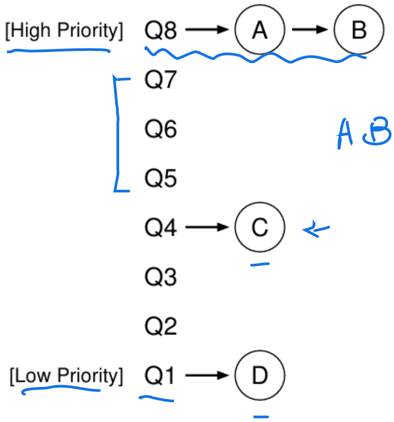
We may favor some processes over others.

Multi-level Feedback Queue (MLFQ)

There are multiple queues with different priority levels.

- If Priority(A) > Priority(B), A runs (B doesn't).
- If Priority(A) = Priority(B), A and B run in RR.

A Snapshop of MLFQ



RR.

A B A B A B C —> (finish) D.

Dynamic Priority

We may lead processes to *starvation* if there's a lot of higher priority processes

One solution is to have the OS dynamically change the priority

Dynamic Priority Attempt 1

New jobs are placed at the top queue with highest priority. (Q8)

Allotment: Each job is allocated a time slice before the scheduler reduces its priority.

eg. 5 unit

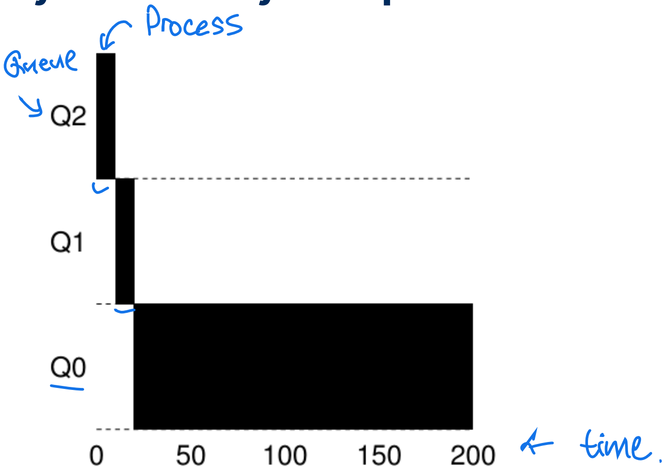
(Q8 → Q7).

If a job uses up its allotment while running, its priority is reduced.

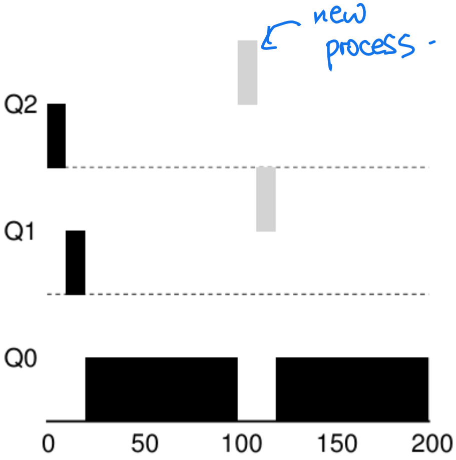
A job's usage resets if it gives up the CPU before its allotted time is used up.

P_0: 3 unit, I/O, 3 units.

Dynamic Priority Attempt 1



Dynamic Priority Attempt 1



Dynamic Priority Attempt 1

Prioritizes I/O-intensive interactive jobs (good response time).

Can cause starvation.

5 units

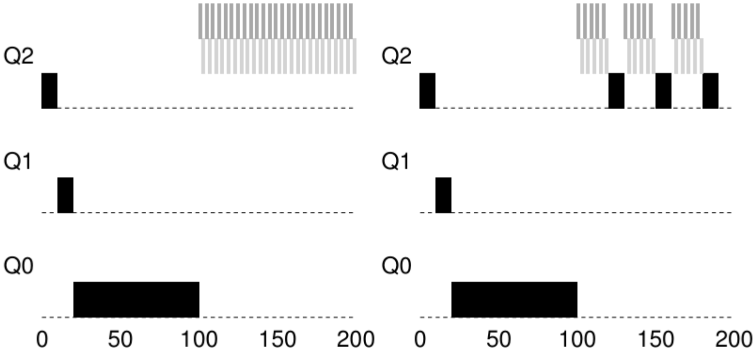
Users can try to cheat the system.

4 units / IO / 4 units

What if a long-running program becomes interactive later?

Dynamic Priority Attempt 2

After some time period S, move all the jobs in the system to the topmost queue.



Dynamic Priority Attempt 3

A job's usage **does not reset** if it gives up the CPU before its allotted time is used up.

How many queues?

How long is an allotment?

How long is the time period S?

Scheduling Can Get Complicated

There's no "right answer", only trade-offs

We haven't talked about multiprocessor scheduling yet

We'll assume symmetric multiprocessing (SMP)
- All CPUs are connected to the same physical memory
- The CPUs have their own private cache (at least the lowest levels)

One Approach is to Use the Same Scheduling for All CPUs

- There's still only one scheduler
  - It just keeps adding processes while there's available CPUs
- Advantages
  - Good CPU utilization
  - Fair to all processes
- Disadvantages
  - Not scalable (everything blocks on global scheduler)
  - Poor cache locality
- This was the approach in Linux 2.4

We Can Create Per-CPU Schedulers

- When there's a new process, assign it to a CPU
  - One strategy is to assign it to the CPU with the lowest number of processes
- Advantages
  - Easy to implement
  - Scalable (there's no blocking on a resource)
  - Good cache locality
- Disadvantages
  - Load imbalance
    - Some CPUs may have less processes, or less intensive ones

We Can Compromise between Global and Per-CPU

- Keep a global scheduler that can rebalance per-CPU queues
  - If a CPU is idle, take a process from another CPU (work stealing)
- You may want more control over which processes can switch
  - Some may be more sensitive to caches
- Use *processor affinity*
  - The preference of a process to be scheduled on the same core
- This is a simplified version of the O(1) scheduler in Linux 2.6

Real-Time Scheduling is Yet Another Problem

- Real-time means there are time constraints, either for a deadline or rate
  - e.g. audio, autopilot
- A hard real-time system
  - Required to guarantee a task completes within a certain amount of time
- A soft real-time system
  - Critical processes have a higher priority and the deadline is met in practice
- Linux is an example of soft real-time

Linux Also Implements FCFS and RR Scheduling

- You can search the source tree: FCFS (`SCHED_FIFO`) and RR (`SCHED_RR`)
- Use a multilevel queue scheduler for processes with the same priority
  - Also let the OS dynamically adjust the priority
- Soft real-time processes:
  - Always schedule the highest priority processes first
- Normal processes:
  - Adjust the priority based on aging

Real-Time Processes Are Always Prioritized

The soft real-time scheduling policy will either be `SCHED_FIFO` or `SCHED_RR`

- There are 100 static priority levels (0—99)

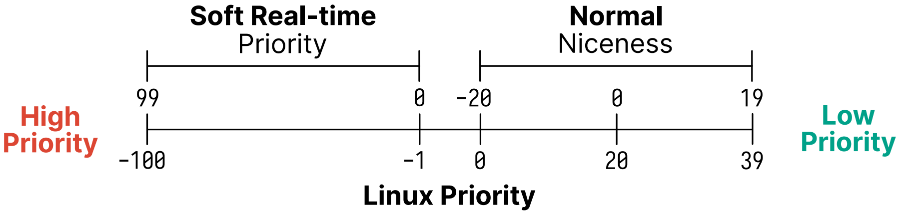
Normal scheduling policies apply to the other processes (`SCHED_NORMAL`)

- By default the priority is 0
- Priority ranges from $[-20, 19]$

Processes can change their own priorities with system calls:

- `nice`, `sched_setscheduler`

Linux Priority Unifies Soft Real-time and Normal Processes



Linux Scheduler Evolution

- 2.4—2.6, a O(N) global queue
  - Simple, but poor performance with multiprocessors and many processes
- 2.6—2.6.22, a per-CPU run queue, O(1) scheduler
  - Complex to get right, interactivity had issues
  - No guarantee of fairness
- 2.6.23—Present, the completely fair scheduler (CFS)
  - Fair, and allows for good interactivity

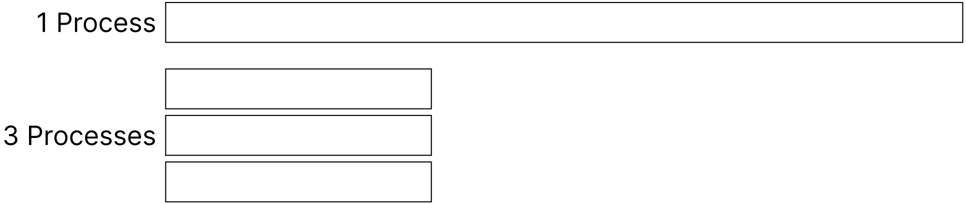
The $O(1)$ Scheduler Has Issues with Modern Processes

- Foreground and background processes are a good division
  - Easier with a terminal, less so with GUI processes
- Now the kernel has to detect interactive processes with heuristics
  - Processes that sleep a lot may be more interactive
    - This is ad hoc, and could be unfair
- How would we introduce fairness for different priority processes?
  - Use different size time slices
  - The higher the priority, the larger the time slice
    - There are also situations where this ad hoc solution could be unfair

Ideal Fair Scheduling

Assume you have an infinitely small time slice

If you have n processes, each runs at $\frac{1}{n}$ rate



CPU usage is divided equally among every process

Example IFS Scheduling

Consider the following processes:

Process	Arrival Time	Burst Time
P_1	0	8
P_2	0	4
P_3	0	16
P_4	0	4

Assume that each vertical slice can execute 4 time units.
Each box represents the time units spend executing

	0				16		24	32
P_1	1	2	3	4	6	8		
P_2	1	2	3	4				
P_3	1	2	3	4	6	8	12	16
P_4	1	2	3	4				

IFS is the Fairest but Impractical Policy

- This policy is fair, every process gets an equal amount of CPU time
  - Boosts interactivity, has the ideal response time
- However, this would perform way too many context switches
- You have to constantly scan all processes, which is $O(N)$

Completely Fair Scheduler (CFS)

- For each runnable process, assign it a "virtual runtime"
  - At each scheduling point where the process runs for time t
    - Increase the virtual runtime by $t \times$ weight (based on priority)
- The virtual runtime monotonically increases
  - Scheduler selects the process based on the lowest virtual runtime
    - Compute its dynamic time slice based on the IFS
- Allow the process to run, when the time slice ends repeat the process

CFS is Implemented with Red-Black Trees

- A red-black tree is a self-balancing binary search tree
  - Keyed by virtual runtime
    - $O(\lg N)$ insert, delete, update, find minimum
- The implementation uses a red-black tree with nanosecond granularity
  - Doesn’t need to guess the interactivity of a process
- CFS tends to favour I/O bound processes by default
  - Small CPU bursts translate to a low virtual runtime
    - It will get a larger time slice, in order to catch up to the ideal